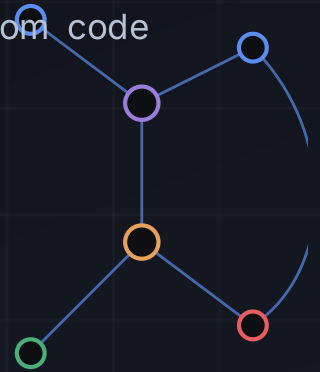


PRODUCT OVERVIEW

Debug AI agents like you debug code.

NodeXLoop is a visual builder, debugger, and governor for agentic software — breakpoints on agent loops, a constitutional guard on every prompt, and a library of governed loop templates from code review to reward training.



01 The idea behind NodeXLoop

AI agents are no longer textboxes — they are loops that observe, plan, call models, and act. The industry's debugging and governance tools were built for requests and responses, not for trajectories. NodeXLoop closes that gap.

A modern agent is a graph: a researcher feeds a writer, a critic scores the draft, a loop controller decides whether to iterate. That graph is where the value lives — and where conventional tooling goes blind. Logs show you requests. They do not show you *which step drifted, what prompt caused it, or why the loop didn't stop*.

NodeXLoop's premise is that agentic software deserves the two things ordinary software has had for decades:

A real debugger.

Breakpoints on nodes, step-through execution, inspection of every prompt and response, replay of failed runs, and side-by-side comparison of two runs to find exactly where their behavior diverged.

A real governor.

A constitutional guard that runs *inside* the executor, not beside it. Every prompt and completion is classified before it moves; every critic iteration is judged for whether it is still making progress; every decision produces a signed manifest.

Both ship in the same surface: a visual graph builder, available as a VS Code extension and in the browser, where the graph you draw is the graph that runs — and the run you watch is the run you can replay, compare, and audit.

Measured, not illustrative

20 → 6

MODEL CALLS ON A
GOVERNED CRITIC LOOP

70%

FEWER CALLS ON
STALLED LOOPS

0

CALLS ON BLOCKED
PROMPTS

15

POLICY PACKS
INCLUDED

Measured against the real built application by the integration test suite: a critic loop whose model returns the same draft every time costs 20 model calls with the guard off and 6 with it on. A prompt-injection input costs zero — preflight blocks it before the first call.

02 Features

The debugger

F9

Breakpoints and stepping

Set a breakpoint on any node. Runs pause there with the exact prompt, context, and response visible — then continue or step, the same workflow every code debugger taught you.

R

Replay and compare runs

Every run is recorded step by step. Replay a failure exactly as it happened, or pin two runs side by side and see the precise step where their trajectories diverge — a diff for agent behavior.

\$

Tokens and cost, per run

Token and cost readouts on the runs screen, taken from the provider's own usage block where one is returned, with per-model \$/1M rates you configure. A readout, not an invoice.

The governor — Axiom runtime guard

■

Preflight classification, offline by default

Prompts and completions are classified before they move, by a guard ported line-for-line from the hosted Axiom engine — pure pattern matching and arithmetic, no network, on by default. Fifteen policy packs included: prompt injection, code review, HIPAA, PCI-DSS, GDPR Article 9, COPPA, SEC 10b-5, and more.

Δ

Loop trajectory governance

The governor tracks two trajectories per loop — commitment (is the reasoning converging?) and risk (is it drifting toward a block basin?) — and stops loops that stall, go flat, or diverge. Cost: zero extra model calls.

✍

Signed manifests, honest escalation

Every decision produces a tamper-evident manifest. Optional escalation sends borderline verdicts to the hosted firewall for a second opinion — a remote block is authoritative, a remote allow never overturns a local block, and if the network fails the local verdict stands. The guard never fails open.

The platform

M

Bring your own model

OpenAI, Anthropic, Google, OpenRouter, Ollama, LM Studio, vLLM, or any OpenAI-compatible endpoint. Built-in models take API keys, can be favoured, and carry a visible badge on each node — amber when a key is missing, before you run.

.lp

Git-native graph definitions

Graphs sync two-way with `.loop.yaml` in your repo — versioned, reviewable, diffable. Per-node `impl: {file, symbol}` bindings jump through the language server, and stale-binding detection flags code that changed out from under a graph.

vs

VS Code native

The builder runs in the sidebar or a full editor panel. API keys live in the OS keychain via SecretStorage, model calls route through the extension host (no browser CORS), workspace files attach to nodes directly, and a host-side network allowlist the web view cannot switch off.

```
# .loop.yaml — an agent your team reviews like code
nodes:
- id: coder
  type: agent
  impl: { file: src/agents/coder.ts, symbol: runCoder }
  permissions:
    files:  ["/src/**"]      # declared scope, recorded in the manifest
    network: deny           # enforced via host allowlist
```

03 Benefits

The features compound into four outcomes that matter to the people paying for, shipping, and auditing agentic systems.

Table 1 What NodeXLoop changes for each audience

BENEFIT	HOW IT SHOWS UP	WHO FEELS IT FIRST
Lower model spend	The governor stops stalled and flat loops early — measured at 70% fewer calls on stuck loops, and zero calls on policy-blocked prompts, which never reach the model at all.	Platform teams watching token bills; anyone on per-call pricing.
Debuggable failures	"The agent did something weird" becomes a paused run, a replay, and a step-level diff between the bad run and the last good one — not a shrug and a rerun.	Engineers on call for agent behavior.
Auditable governance	Every guard decision produces a signed manifest; blocks halt a branch rather than a run; policy packs for HIPAA, PCI-DSS, GDPR Art. 9, COPPA and SEC 10b-5 run locally, offline.	Regulated industries, compliance reviewers.
No lock-in	The same graph runs against any provider or local model; governance and debugging operate on the trajectory, not the vendor. Graphs live in your git repo, not a dashboard.	Teams with mixed model fleets or self-hosted weights.
Reviewable autonomy	Agents are declared in <code>.loop.yaml</code> , bound to real code via <code>impl</code> , and flagged when bindings go stale — agent definitions go through the same PR review as everything else.	Engineering leads, security review.

Why "governed" beats "guarded"

Point solutions filter individual prompts. NodeXLoop governs *trajectories*: a sequence of perfectly legal calls can still be a runaway loop, a drifting agent, or a reward hack in progress. Watching the shape of behavior over time — not just the events — is what catches those.

04 The loop library

NodeXLoop ships pre-built loop graphs for common workflows — and a growing set of advanced templates that encode hard-won lessons about agentic control: held-out judges, authority gates, and human-promoted skills.

Core workflows

Table 2 Built-in starter templates

TEMPLATE	FLOW
Blog post factory	Research → Write → Critic → Publish, iterating until the quality threshold is met.
Code review cycle	Lint → Test → Review → Fix — automated PR review with an iterative feedback loop.
Trip planner	Parallel search over flights, hotels and activities, merged into a confirmed itinerary.
Research synthesizer	Multi-source research with cross-referencing and fact-checking loops.
Landing page builder	Copy → Design → Code → Preview, iterating on layout and messaging.
Data pipeline	Extract → Transform → Validate → Load, with error recovery loops.

Advanced and experimental loops

Agent failure triage

[Reproduce](#) → [Localize](#) → [Patch](#) → [Prove](#) ↻

Turns "the agent did something weird" into a named cause and a test that fails before the fix and passes after it. The loop does not close until the proof passes — patching without proof just iterates.

Reward loop with a held-out judge

EXPERIMENTAL

[Rollout](#) → [Reward](#) → [Update](#) → [Evaluate](#) ↻

An RL-style training loop for agent policies — with the crucial separation made structural: **the scorer that trains the policy is never the scorer that promotes it**. A held-out judge evaluates candidates the training reward never saw, so reward hacking shows up as a visible gap between the two scores instead of a silently corrupted policy.

Robot task authority

[Perceive](#) → [Plan](#) → [Authority gate](#) → [Act](#) → [Review](#) ↻

Physical-world agent control: the plan is checked against the robot's trust level before a motor turns, and every refusal is recorded. Pairs with the Fleet sandbox, where a dozen simulated robots move through trust-level zones under graduated constitutional responses — and any incident exports as a NodeXLoop graph for replay in the builder.

Skill acquisition pipeline

EXPERIMENTAL

[WATCH](#) → [LEARN](#) → [OPTIMIZE](#) → [GOVERN](#) ↻

An agent learns a reusable skill from a signed demonstration trace, an optimizer loop finds a leaner version — and nothing becomes usable until a named human promotes it. Optimized versions return to pending review; a skill cannot self-certify.

05 RL-style loops, without the usual failure modes

Three of the library's loops borrow from reinforcement learning — rollout, reward, update, evaluate — and rebuild the pattern so its classic failure modes become visible instead of silent.

The failure mode that matters: reward hacking

Any loop that optimizes against a score will eventually find ways to raise the score without doing the work. In classical RL this is reward hacking; in agentic systems it looks like a critic that rubber-stamps, an evaluator that shares blind spots with the worker, or a policy that overfits to its own grader. The common root cause is structural: **the same signal that trains is the signal that promotes.**

The held-out judge pattern

Rollout generates candidate behavior. A training reward scores it and drives updates. But promotion decisions come from a *separate* judge — held out from the update loop, evaluating on cases the reward never optimized against. If the policy learns to game the trainer, the two scores diverge, and the gap is exactly what you inspect. A divergence you can see is a reward hack you can catch.

Why the governor is load-bearing here

Self-improvement loops are where runaway cost and silent drift do the most damage — which is precisely what the Axiom guard watches:

Table 3 Guardian rules applied to training-style loops

STOP CONDITION	WHAT IT CATCHES IN AN RL-STYLE LOOP
LOOP_STALLED	Candidate drafts stop changing — the update step is no longer producing novel behavior.
LOOP_FLAT	Score gains fall below threshold, or any regression — training has converged or is thrashing.
LOOP_DIVERGENT	The commitment trajectory turns non-monotonic — successive iterations are un-committing, a signature of an unstable update.
CONSTITUTIONAL_DRIFT	Cumulative drift reaches BLOCK — the loop is optimizing toward territory policy forbids. Halts the branch, not the run.
BUDGET_EXHAUSTED	Run-level step or wall-clock budget — the hard cap on what a self-improvement loop may spend.

How the three loops compose

The RL-flavored templates are not three curiosities — they are one pattern at three altitudes, all sharing the same governance spine:

Table 4 One pattern, three altitudes

LOOP	WHAT'S LEARNED	THE HELD-OUT / AUTHORITY CHECK	ALTITUDE
Reward loop	A policy	Held-out judge scores what the trainer never saw	Model behavior
Skill acquisition	A reusable procedure	A named human promotes; skills cannot self-certify	Tool-call sequences
Robot task authority	A physical plan	Trust-level gate before any motor turns	The physical world

In each case the loop is allowed to *propose* autonomously and required to *promote* through a check the loop itself does not control. That asymmetry — autonomous optimization, gated adoption — is the design center of NodeXLoop's approach to self-improving systems.

06 Straight talk: current limits

We would rather you hear the edges from us. Four things, plainly:

1. Permissions are declared, not enforced — with one exception.

The `.loop.yaml` permission block is a reviewable contract recorded into signed manifests. The exception is network: enforced as a host allowlist in the extension host. File, shell, git, and secrets permissions are the schema to enforce against once those node types exist — the manifests say "declared", and we do not claim otherwise.

2. It doesn't generate graphs from your code.

NodeXLoop won't scan a codebase and produce an agent graph. What it does is detect stale bindings — when code changes out from under an existing graph — so drift between code and definition is visible instead of silent.

3. Cost figures need your rates.

The measured reductions — 70% fewer calls on stalled loops, $20 \rightarrow 6$ on a governed run — are call counts. Dollar savings depend on your model and pricing; per-model rates in the registry make the runs-screen figure yours to trust.

4. The experimental loops are experimental.

The reward loop and skill acquisition templates encode the right structure — held-out judges, human promotion — but the model-backed generalization and rewriting stages behind them are documented seams, not shipped machinery. They are starting points you can run, inspect, and harden.

Working with NodeXLoop

- **Run it in the browser** — nodexloop.orivael.dev
- **Run it in VS Code** — search "NodeXLoop" in the marketplace; keys stay in your OS keychain
- **Follow the open work** — github.com/Orivael-Dev/axiom
- **Talk to us** — hello@orivael.dev

The short version

NodeXLoop is the debugger and governor for agentic software: breakpoints, replay, and run comparison; a constitutional guard that classifies every prompt offline and stops runaway loops at a measured 70% fewer wasted calls; git-native graph definitions; any model you bring. And a loop library that extends the same governed pattern all the way to reward training, learned skills, and physical robots — always with a check the loop itself cannot overrule. **NodeXLoop — an Orivael product. Governed by Axiom.**